

Conception d'une plateforme de traitement de données sportives

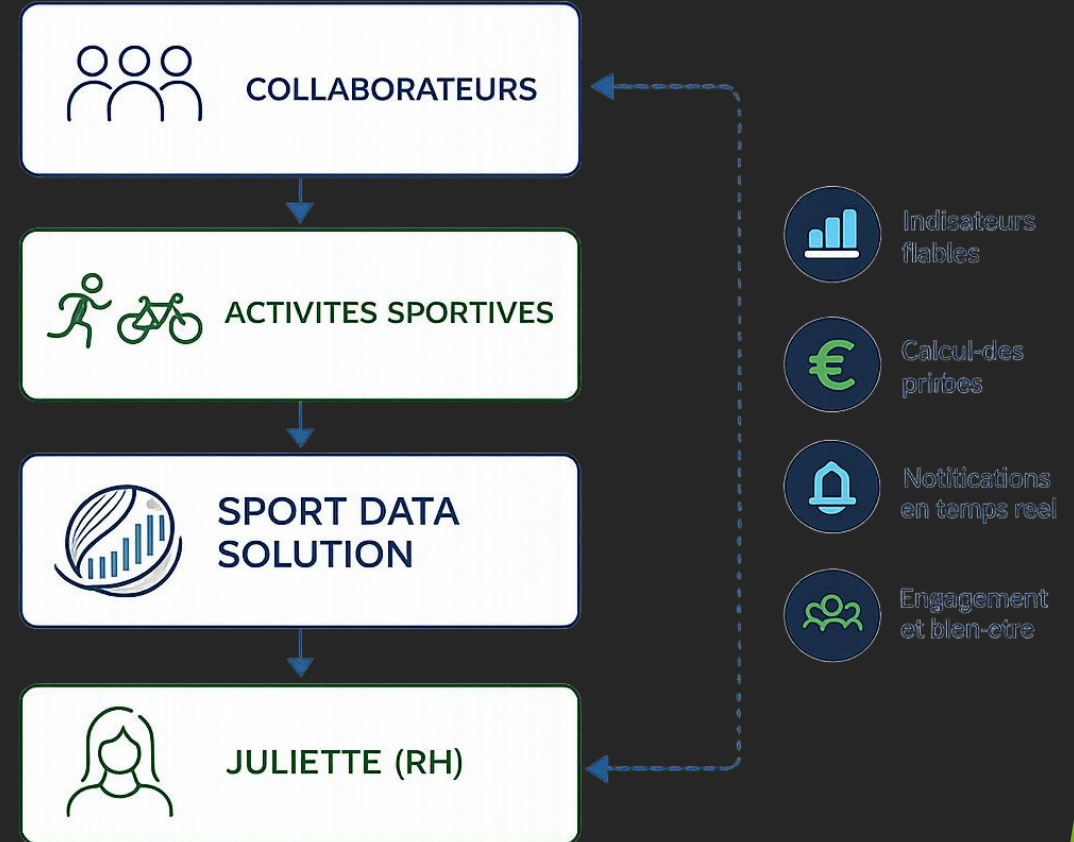
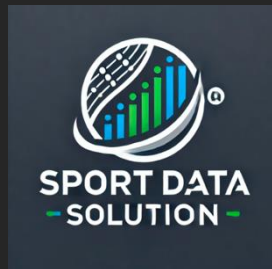
Projet 12 – Parcours Data Engineer – OpenClassrooms
Sokhna Signara Gueye
Juillet 2026

Contexte

Juliette, responsable RH, souhaite mettre en place un programme de bien-être afin d'encourager l'activité physique des collaborateurs.

Pour accompagner cette initiative, elle souhaite disposer d'une plateforme capable de centraliser ces activités afin de :

- ▶ Suivre la participation des salariés
- ▶ Calculer automatiquement les primes associées
- ▶ Produire des indicateurs RH fiables
- ▶ Notifier les nouvelles activités en temps réel



Objectifs du projet

Automatiser la collecte

Collecter automatiquement les activités sportives des collaborateurs via une API.

Garantir la qualité des données

Mettre en place des contrôles automatiques pour assurer des données fiables.

Produire des indicateurs

Calculer les primes et fournir des indicateurs pertinents pour les équipes RH.

Automatiser le reporting et les alertes

Mettre à disposition des tableaux de bord et notifier les nouvelles activités.

Orchestration

Pourquoi Kestra ?

- ✓ Automatisation complète du pipeline
- ✓ Orchestration des différentes briques techniques
- ✓ Exécution séquentielle des traitements
- ✓ Suivi visuel de chaque exécution
- ✓ Gestion des erreurs et historique des workflows



←

→

↻

localhost:8080/ui/main/executions/sport/generate-consume-transform-quality/7XW0ICpEpRI9on30wg3Mdv/gantt?filters[level][EQUALS]=IN...

☆

🔥

W

ABP

★

📦

↓

🔄

S

Redémarrer pour mettre à jour

🏠

kestra.

🤖 AI Copilot

📊 Dashboards

📁 Flows

📱 Apps

▶ Executions

📄 Logs

🧪 Tests

📦 Assets

📁 Namespaces

⚙️ Plugins

📋 Blueprints

📅 Tenant

🖨️ Instance

🏠 Kestra

Executions / sport.generate-consume-transform-quality

7XW0ICpEpRI9on30wg3Mdv

🌟

✎ Edit flow

▶ Execute

Overview

Gantt

Logs

Topology

Outputs

Metrics

Dependencies

Audit Logs

Assets

🔍 Add filters

🔍 Search executions

Log Level INFO

✕ Clear all

🔄 Refresh data

📄 Saved filters

0

⌵

⌵

🔍

Periodic refresh

	16.60s	9:31:08	9:31:11	9:31:14	9:31:17	9:31:21
> generate_daily						
> consume						
> transform						
> quality						

Orchestration (automatisation)

Orchestration (automatisation)

The screenshot displays the Kestra orchestration platform interface. The left sidebar contains navigation links for AI Copilot, Dashboards, Flows, Apps, Executions, Logs, Tests, Assets, Namespaces, Plugins, Blueprints, Tenant, and Instance. The main panel shows the 'generate-consume-transform-quality' flow under the 'sport' namespace. The 'Triggers' tab is active, displaying a table with one trigger: 'daily_schedule' of type 'io.kestra.plugin.core.trigger.Schedule', with a next execution date of 'Fri, Jul 31, 2026 8:00 AM'. Below the table is a 'Logs' section with a bar chart titled 'Logs count per date grouped by level'. The chart shows two bars for the dates '2026-07-29' and '2026-07-30', both with a height of 4. The bottom section shows a log of three debug messages, each representing a task execution (quality, transform, consume) with a response code of '200'.

Id	Type	Next execution date	Backfill
daily_schedule	io.kestra.plugin.core.trigger.Schedule	Fri, Jul 31, 2026 8:00 AM	Backfill executions

Logs
Logs count per date grouped by level

DEBUG

Total Executions

Execution Date

2026-07-29

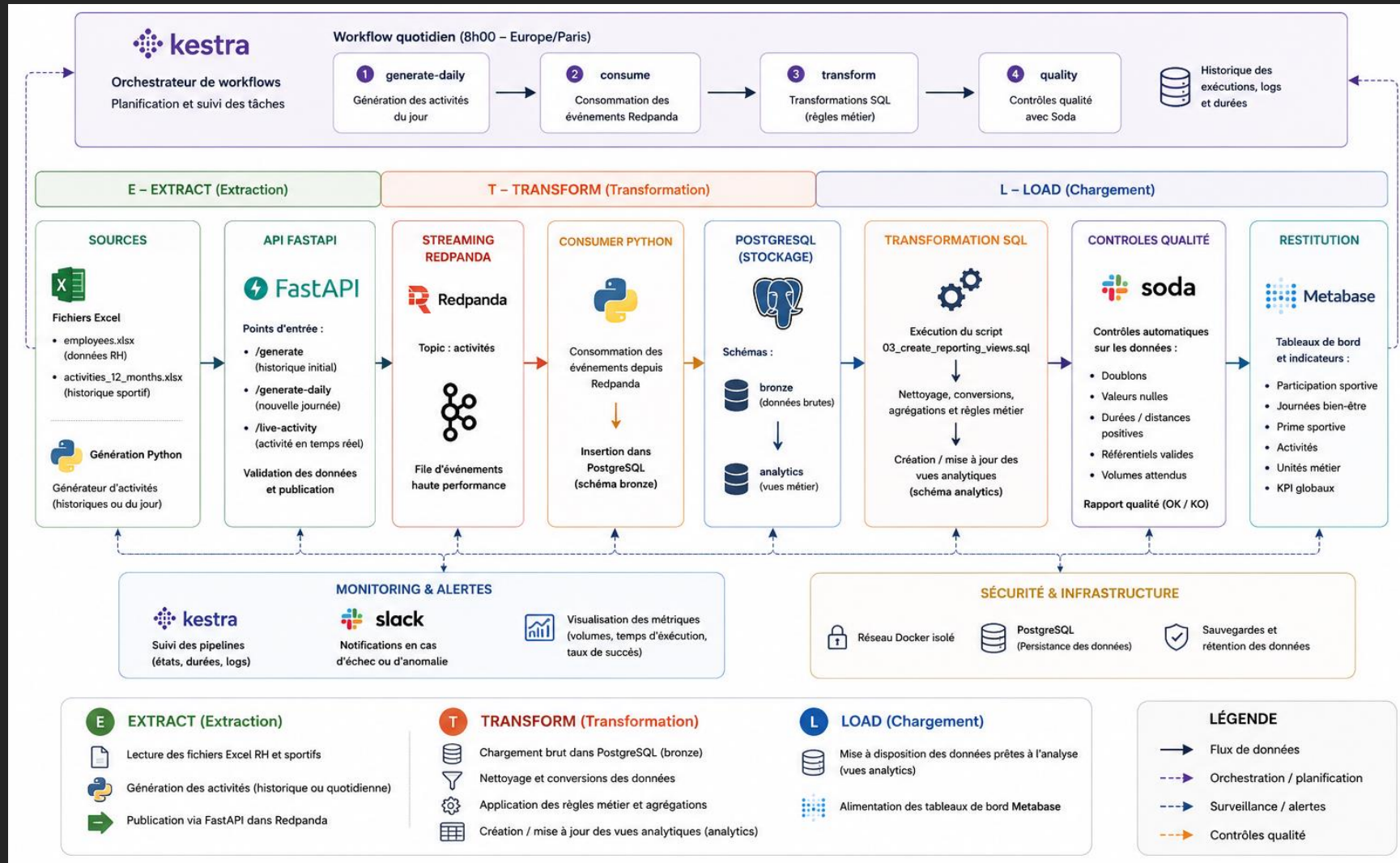
2026-07-30

DEBUG 2026-07-30 21:31:24.578 taskId: quality executionId: 7XW0ICpEpRI9on30wg3Mdv
Request 'http://sport_app:8000/quality' from 'sport_app/172.20.0.5:8000' with the response code '200'

DEBUG 2026-07-30 21:31:22.982 taskId: transform executionId: 7XW0ICpEpRI9on30wg3Mdv
Request 'http://sport_app:8000/transform' from 'sport_app/172.20.0.5:8000' with the response code '200'

DEBUG 2026-07-30 21:31:22.657 taskId: consume executionId: 7XW0ICpEpRI9on30wg3Mdv
Request 'http://sport_app:8000/consume' from 'sport_app/172.20.0.5:8000' with the response code '200'

Architecture de la solution



Démonstration

Objectif : démontrer le fonctionnement complet du pipeline de données.

Étapes de la démonstration :

Modification du taux de prime dans la base de données.

Ajout d'une nouvelle activité sportive via l'API FastAPI.

Publication de l'événement dans Redpanda.

Consommation et chargement dans PostgreSQL.

Contrôle qualité automatique avec Soda.

Actualisation des indicateurs dans Metabase.

Notification en temps réel dans Slack.

Démonstration

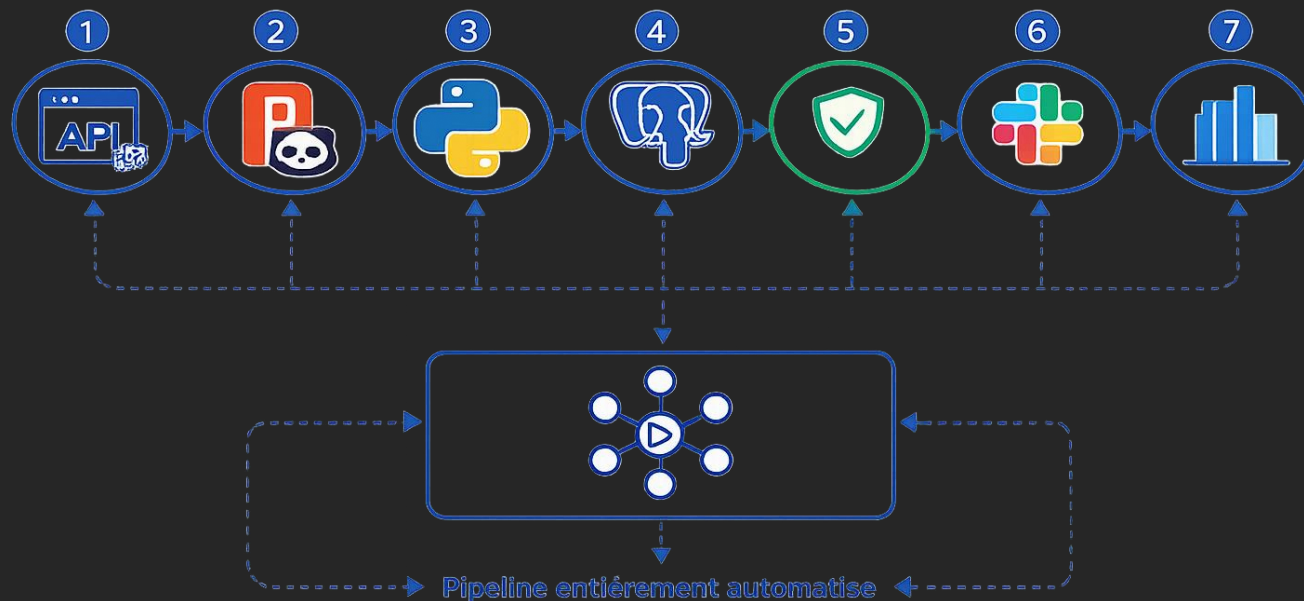
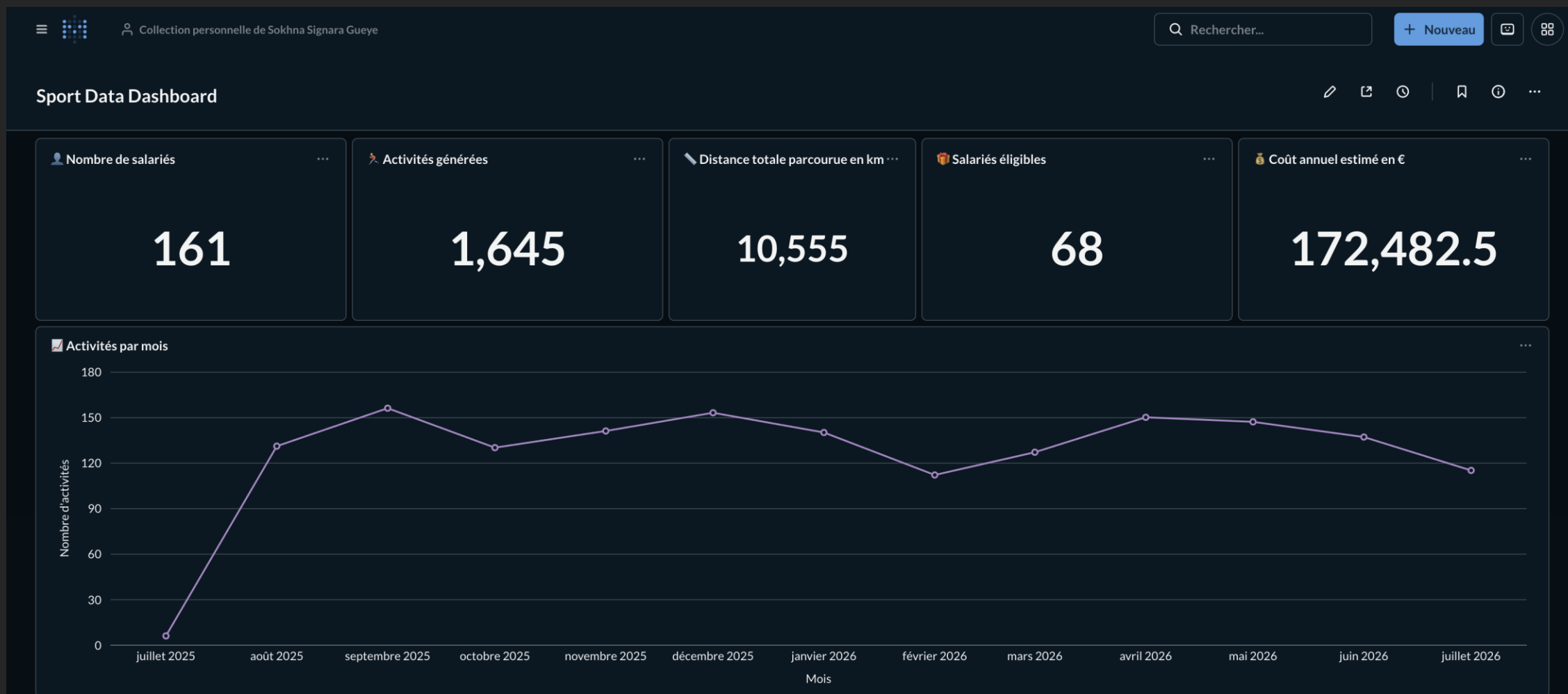
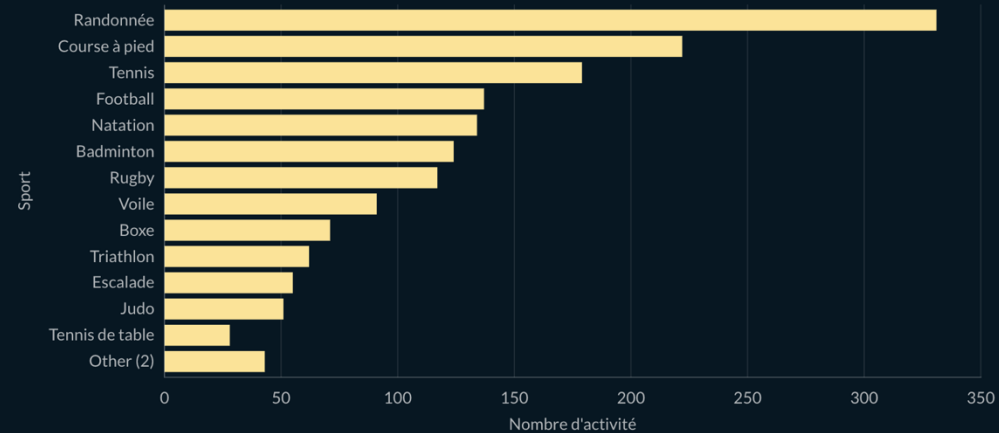


Tableau de bord décisionnel

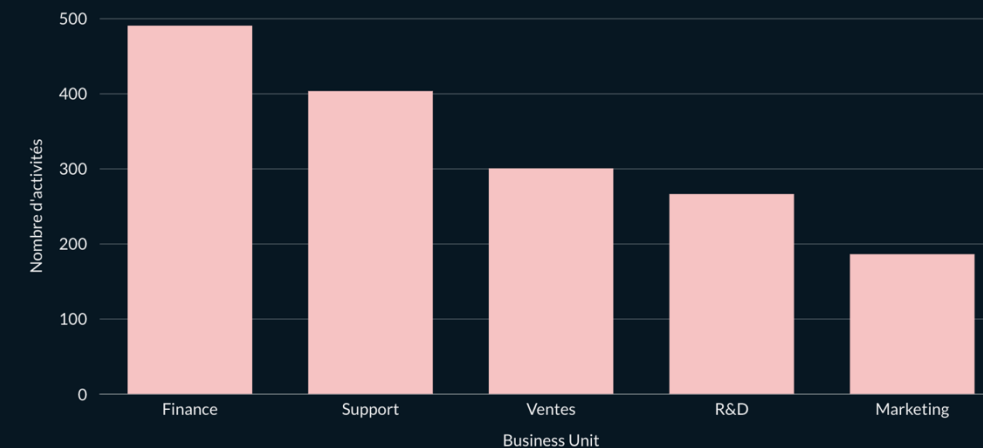




🏆 Répartition des activités par sport



Activités par BU



👤 Top 10 des salariés les plus actifs

Nom de du salarié	Nombre d'activités	Distance parcourue en km	Durée de pratique en heures
Sabine Marchand	45	591.51	133.51
Marine Gautier	45	81.28	32.68
Jeremy Arnaud	41	0	63.27
Claudine Joubert	40	0	61.92
Philippine Boutin	38	0	59.67
Hugues Fischer	38	568.3	136.19
Bernard Georges	35	537.76	125.36
Susanne Lacroix	35	0	46.47
Claire Rossi	35	0	55.6
Alfred Meyer	35	0	60.93

Choix techniques

Besoin	Solution	Justification
API	FastAPI	API légère, rapide et documentée
Streaming	Redpanda	Architecture évènementielle
Base de données	PostgreSQL	Fiabilité et SQL
Orchestration	Kestra	Automatisation des traitements
Qualité	Soda	Contrôles automatiques
Reporting	Metabase	Visualisation simple
Notifications	Slack	Alertes en temps réel

Difficultés rencontrées

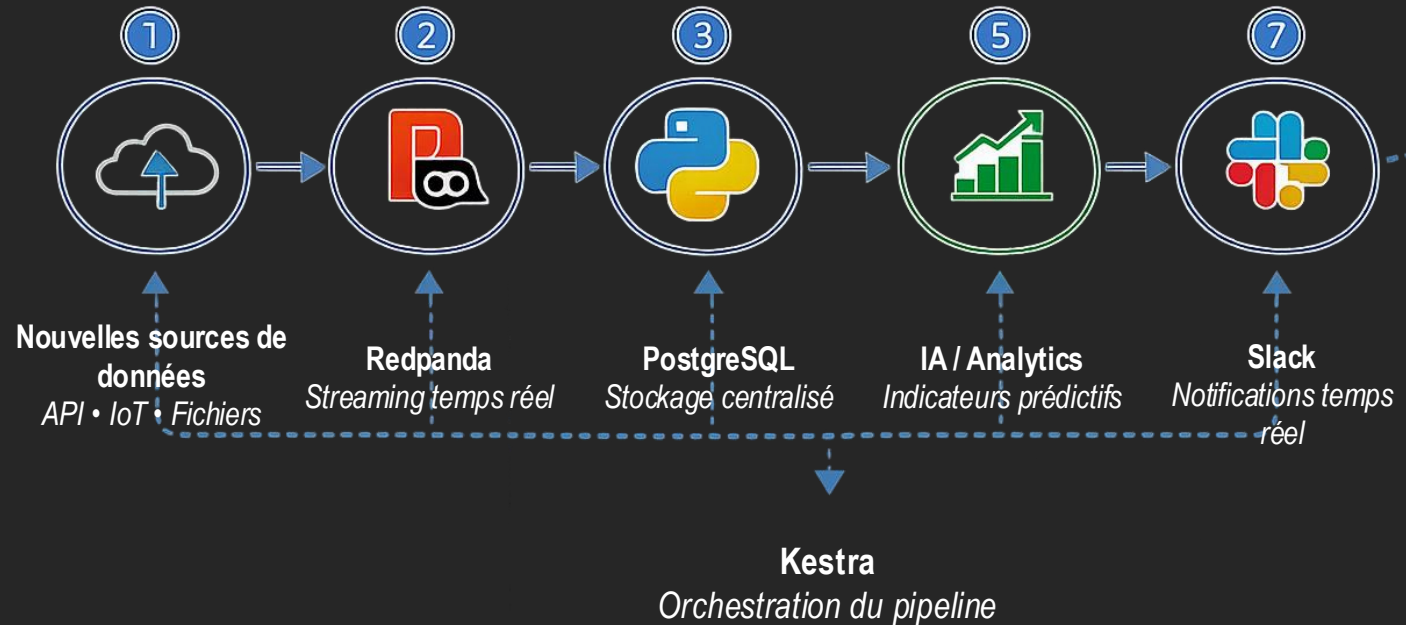
Les difficultés auxquelles j'ai été confrontées :

- ▶ Communication entre conteneurs Docker
- ▶ Conflit de ports
- ▶ Intégration Soda
- ▶ Volumes Docker

Ce que j'ai mis en place pour palier à ces difficultés :

- ▶ Architecture Docker
- ▶ Orchestration
- ▶ Contrôle qualité
- ▶ Pipeline événementiel

Conclusion



Aujourd'hui

- Pipeline automatisé
- Qualité des données
- Dashboard
- Notifications

Demain

- Nouvelles sources
- Objets connectés
- Cloud
- Authentification
- Indicateurs avancés

Pipeline entièrement automatisé